

Technical Report: Web-Based Othello Application with WebAssembly AI

System Architecture & Logic Analysis

May 20, 2026

Abstract

This report details the technical architecture, algorithms, and execution flows of a web-based Othello (Reversi) application. The system is designed with a strict separation of concerns, utilizing a Canvas-based rendering engine for the user interface, a Web Worker for asynchronous game state management, and a WebAssembly (Wasm) module for high-performance AI calculations. The application dynamically fetches and decompresses gzipped evaluation heuristics and opening books, mapping them into the Wasm memory. This report dissects the module interactions, the asynchronous request-response lifecycle, the rendering logic, and the bitboard-like state representation used to facilitate the game logic.

1 Introduction

Othello is a deterministic, two-player strategy game with a branching factor that demands efficient computational evaluation for AI opponents. Implementing such an AI in a web browser poses challenges regarding main-thread blocking and execution speed.

This application addresses these challenges through a modern web architecture:

- **Performance:** Core AI logic is compiled to WebAssembly (Wasm), executing at near-native speeds.
- **Responsiveness:** The Wasm module runs inside a Web Worker, preventing UI freezes during deep tree searches.
- **Asynchronous Communication:** A custom Promise-based RPC wrapper facilitates communication between the main thread and the worker.
- **Rich UI:** HTML5 Canvas API is used for immediate-mode rendering of the board, stones, legal moves, and evaluation scores.

2 System Architecture

The system is decomposed into distinct modules, each responsible for a specific domain logic.

2.1 Frontend Rendering Layer

Comprises `board.js`, `status.js`, and the UI class in `game.js`. This layer is strictly responsible for visual representation and DOM event capturing. It uses an immediate-mode rendering paradigm where the entire canvas is cleared and redrawn on every state update.

2.2 Orchestration Layer

The `Game` and `EventDispatcher` classes in `game.js` act as the mediator. The `EventDispatcher` implements the Observer pattern to decouple UI events (clicks, button presses) from game logic, while the `Game` class manages the high-level game loop, turn switching, and AI triggering.

2.3 Communication Layer

The `WorkerWrapper` and `Engine` classes in `game.js` abstract the Web Worker API. They implement an asynchronous request-response pattern using a `Map` of pending Promises, keyed by a unique `requestId`.

2.4 Computation Layer (`engine.js` & `othello.js`)

Runs entirely in a Web Worker. `engine.js` initializes the Wasm environment, fetches external data files, and routes incoming messages to the Wasm App. `othello.js` is the auto-generated Wasm glue code handling memory allocation, garbage collection, and function proxying.

3 Core Algorithms & Data Structures

3.1 State Representation

The game state is transferred between the Wasm module and the JavaScript layer via a JSON-like object. Bitboards are serialized as 64-character strings of '1's and '0's. Notably, the strings are reversed (LSB first) before transmission:

- `black`: 64-character string representing black stones.
- `white`: 64-character string representing white stones.
- `legal_moves`: 64-character string representing valid moves for the current player.
- `eval`: Array of integers representing the heuristic score of each legal move.

The string reversal (`split("").reverse().join("")`) aligns the Wasm internal bitboard memory layout (where index 0 often corresponds to the top-left or bottom-right corner depending on endianness) with the UI coordinate system.

3.2 AI and Evaluation Heuristics

The AI operates on a Minimax algorithm with Alpha-Beta pruning, encapsulated within the Wasm binary. The search depth is dynamically adjustable via a level slider (1 to 32).

- **Evaluation Data:** Fetched as `eval1.json.gz` and `eval2.json.gz`. These contain pre-calculated positional weights and mid-game heuristics.
- **Opening Book:** Fetched as `opening.txt`. Contains deterministic move sequences. The UI provides a mapping of 294 specific opening strategies (e.g., " ", " ") which can be forced upon the game state using `set_human_opening`.

3.3 Asynchronous RPC Mechanism

Since Web Workers communicate via synchronous string/serialized message passing, the application implements an asynchronous RPC mechanism:

1. Main thread calls `Engine.put(pos)`.
2. `WorkerWrapper.sendRequest` generates a unique `requestId` and creates a Promise.
3. The Promise resolver is stored in a Map keyed by `requestId`.
4. A message $\{type : 'put', payload : [pos], requestId : id\}$ is posted to the worker.
5. The worker executes the Wasm function and posts back $\{payload : result, requestId : id\}$.
6. The worker's `onmessage` handler resolves the pending Promise, removing it from the Map.

4 Execution Flows

4.1 Initialization Flow

The bootstrapping process requires careful synchronization of asynchronous tasks:

Algorithm 1 System Initialization

- 1: Main thread instantiates `Game` and `Engine` (spawns Worker).
 - 2: Worker imports `__wbg_init` (Wasm glue).
 - 3: Worker asynchronously fetches and decompresses `eval1.json.gz` and `eval2.json.gz`.
 - 4: Worker fetches `opening.txt`.
 - 5: Worker instantiates Wasm App with eval and opening data.
 - 6: Worker posts "ready" to main thread.
 - 7: Main thread receives "ready", enables UI, and calls `draw_force()`.
-

4.2 Game Move Flow

When a user clicks a valid position on the board:

Algorithm 2 User Move Execution

```
1: Event listener maps canvas coordinates to board index  $p$ .
2: Game.boardClick(p) sets isThinking = true.
3: Game.put(p) is invoked.
4: if isPass() then
5:   Display "Pass" overlay for 1000ms; execute engine.pass().
6: end if
7: Retrieve state via engine.getState(null).
8: if AI enabled and current turn matches AI turn then
9:   Execute aiPut() loop.
10: else
11:   Execute engine.put(p).
12: end if
13: Force redraw, check for game end.
14: Set isThinking = false; trigger background evaluation draw.
```

4.3 Undo Logic

The undo logic must account for the AI's turns. A single undo should revert the board to the previous state where it was the human's turn.

Algorithm 3 Undo Operation

```
1:  $max\_iterations \leftarrow 70$ 
2: while  $max\_iterations > 0$  do
3:   Get current state; count stones.
4:   if Stone count == 5 and AI plays Black then
5:     break
6:   end if
7:   engine.undo()
8:   Get new state.
9:   if Next turn is Human and not Pass then
10:    break
11:   end if
12:    $max\_iterations \leftarrow max\_iterations - 1$ 
13: end while
```

5 WebAssembly Integration (othello.js)

The `othello.js` file acts as the host binding for the Wasm module. It manages:

- **Memory Management:** It exposes `__wbindgen_malloc` and `__wbindgen_free` to pass strings between JS and Wasm. Strings are encoded as UTF-8, written to Wasm memory, and pointers are passed to the Wasm functions.

- **Garbage Collection:** It utilizes a `FinalizationRegistry` to automatically call Wasm destructors (`__wbg_app_free`) when JavaScript garbage-collects the `App` wrapper object.
- **Closure Handling:** It creates `MutClosure` objects to pass JavaScript functions (like Promises and event callbacks) into Wasm, managing their reference counts to prevent premature collection.

6 Rendering System

The rendering engine utilizes the HTML5 Canvas 2D API. The layout is calculated based on a virtual board size of 540x540 within a 600x800 canvas.

6.1 Coordinate Mapping

Board cells are mapped to screen coordinates using the formula:

$$X_{coord} = X_{offset} + Padding + i \times (CellSize + CellMargin) \quad (1)$$

Click events are reverse-mapped using:

$$Col = \lfloor \frac{X_{click} - X_{offset} - Padding}{CellSize + CellMargin} \rfloor \quad (2)$$

The application handles responsive scaling by comparing window dimensions to the base canvas size (600x800) and applying CSS width/height scaling, while keeping the internal rendering resolution fixed.

6.2 Immediate Mode GUI

The `update()` method clears the specific canvas regions and redraws:

1. Background and grid tiles.
2. Coordinate labels (A-H, 1-8).
3. Legal move highlights (Cyan tiles).
4. Human opening next position highlights (Black tiles).
5. Stones with shadow effects.
6. Evaluation scores overlay (Blue for max score, White for others).
7. Status UI (Scores, player names, control buttons).

7 Conclusion

The application represents a robust implementation of a classic board game utilizing modern web standards. By offloading the intensive Minimax search to a pre-compiled WebAssembly module running in a Web Worker, the application achieves high AI performance without degrading the user interface responsiveness. The custom RPC mechanism over Web Workers, combined with efficient bitboard state serialization, provides a seamless bridge between the DOM UI and the Wasm computation core.

References

- [1] Haas, A., et al. (2017). *Bringing the Web up to Speed with WebAssembly*. Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation.
- [2] Rosenbloom, A. (1982). *A World-Championship-Level Othello Program*. Artificial Intelligence, 19(3), 279-293.
- [3] MDN Web Docs. *Web Workers API*. Mozilla. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API
- [4] MDN Web Docs. *Canvas API*. Mozilla. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
- [5] Rust and WebAssembly Working Group. *wasm-bindgen: Facilitating high-level interactions between Wasm modules and JavaScript*. Retrieved from <https://github.com/rustwasm/wasm-bindgen>